

Prompt Receipts

From Prompt Strings to Reconstructable Execution Events

A paper assembled from the Prompt Practices research field

Abstract. Prompt research still often treats the prompt as a string: a visible linguistic object that can be copied, compared, improved, and archived. That unit is increasingly inadequate. Empirical work on Midjourney communities shows that prompts acquire social histories through reuse, collaboration, provocation, and disputes over ownership. Work on distributed cognition suggests that external products of earlier activity can transform later cognition. Current agent systems add a second displacement: the text a user writes may be only one component of a larger execution context assembled from instructions, tools, history, files, and state. Model versions, personalization, and other execution conditions can change while the visible string remains fixed. At the same time, production guidance increasingly treats prompt behavior as something tested, versioned, routed through repository knowledge, reconstructed from external state, or generated from higher-level descriptions.

This paper argues that the minimal research object for reproducible prompt practice should therefore shift from the *prompt string* to a *prompt receipt*: a bounded record of an execution event and its relevant lineage. A receipt does not claim perfect capture of hidden platform state. It records what can be warranted: visible input, referenced instructions and history, tools, model/version and exposed settings, output, lineage, evaluation, and the evidence motivating later changes. The proposal yields a second design law: archives should preserve more than active contexts load. The archive and the running prompt must diverge, with retrieval becoming an explicit part of prompt practice. The paper develops the argument, names its limits, and proposes empirical tests for receipt sufficiency, epistemic provenance, version drift, and context-recall failure.



1 The prompt string has become a misleadingly small object

Prompting is commonly represented by a simple diagram: a person writes a string, a model receives it, and an output appears. That diagram is useful for describing a single interface gesture. It is increasingly poor at describing the research object that must be preserved if another person is expected to understand, reproduce, or extend the work.

The first pressure on the string comes from social practice. Mahdavi Goloujeh, Sullivan, and Magerko’s study of nineteen Midjourney community members explicitly reframes prompting as a socially situated practice rather than only an individual technique. Their participants describe proprietary, derivative, collaborative, and provocative orientations to prompt sharing and reuse [1]. The categories matter because they show that a string can arrive already entangled in a history: borrowed modifiers, copied structures, shared experiments, attempts to influence a community, and changing judgments about whether reuse counts as learning or appropriation. The visible prompt does not contain that history, but the history can change what the prompt means socially and what later users can learn from it. This is the first displacement developed in SCGAI-001, SCGAI-002, SCGAI-003, and SCGAI-008.

A second pressure is temporal. Hollan, Hutchins, and Kirsh argue that cognitive processes can be distributed through time when products of earlier events alter the conditions of later events [2]. Applied cautiously to prompting, this suggests that prompt history may be more than documentation. A failed image, a discarded phrase, a comparison between two generations, or a remembered branch can change the computation a person performs when composing the next instruction. The final string is then an endpoint of a cognitive trajectory whose earlier artifacts remain functionally relevant. This is the wager in SCGAI-003-A: history can be state, not merely a log.

A third pressure comes from the architecture of current agent systems. In OpenAI’s account of the Codex agent loop, the end user’s visible message is not necessarily the model input verbatim; the system orchestrates model calls with instructions, tool definitions, prior interaction items, and tool results [8]. The significance is not that one implementation is universal. It is that a practical category called “the prompt” now risks conflating two different objects: the *visible user utterance* and the *effective execution context*. When these diverge, textual preservation of the former cannot by itself identify the latter. WWP-20260817-01 formalizes this as

```
EXECUTED_CONTEXT = C(user text, history, tools, instructions, environment, state).
```

A fourth pressure is environmental drift. Midjourney’s documentation exposes model version selection and records changes in default versions [16]. DeVito’s work on adaptive folk theorization provides a useful neighboring concept: when opaque platforms change, users must revise causal understandings that guide their behavior [4]. Together these sources make a precise problem visible. A prompt can remain textually identical while the execution system changes. The preserved string remains the same artifact; the operative practice does not. SCGAI-004-B calls the resulting stale rule a *prompt fossil*: a technique whose words continue to circulate after the system conditions that made it effective have disappeared.

These four pressures do not merely say that “context matters.” They identify different causal locations that a string-only archive collapses:

1. **Social lineage:** where a form came from, how it was reused, and what community norms surrounded that reuse.
2. **Temporal lineage:** the sequence of outputs, failures, comparisons, and revisions that trans-

formed later work.

3. **Execution assembly:** instructions, tools, files, history, and other state present when the model actually ran.
4. **Execution environment:** model/version, exposed settings, personalization, platform, and other conditions under which the string was interpreted.

The prompt string is therefore not false. It is incomplete in a systematic way. It records the most visible part of an execution while omitting several other parts that can explain why the execution behaved as it did and why the next prompt changed.

2 From prompt to prompt receipt

The central proposal of this paper is deliberately modest: replace the string as the default archival unit with a *prompt receipt*. The term is compiler synthesis from this research field, not terminology borrowed from any cited source. A receipt is a bounded record of a prompt execution whose purpose is reconstruction, comparison, and continuation.

The word *receipt* is useful because it avoids two impossible promises. It does not imply that the complete internal state of a proprietary model has been captured. It does not imply that the receipt itself is the program. A receipt is evidence that an execution occurred under a describable set of accessible conditions. Like an experimental lab record, it is valuable to the extent that it makes consequential conditions inspectable and omissions explicit.

A provisional receipt schema from FIELD-20260817-01 is:

Receipt field	What it preserves
Visible input	The text, image, audio, or other user-facing material intentionally submitted.
Instruction-set reference	A hash, version, or address for system/developer/project instructions when inspectable and permissible to preserve.
History reference	The prior conversational or workspace state materially available to the execution.
Tool-schema reference	Which callable tools and schemas were exposed.
Environment reference	Files, repository state, state handles, retrieval results, or other addressable external conditions.
Model/version	The declared model or version when known.
Exposed parameters	Settings, personalization, seeds, reference codes, or other user-visible controls.
Platform and time	Where and when the event occurred.
Output reference	The artifact or response produced.
Lineage references	Parent prompts, source prompts, forks, predecessors, or community artifacts that materially informed the execution.
Evaluation	What counted as success, failure, surprise, or useful evidence.
Change rationale	Why a subsequent mutation was made and what evidence motivated it.

The schema should not be read as mandatory bureaucracy. Receipt sufficiency is relative to the reconstruction question. If the question is whether a visual style survives a Midjourney model

migration, model version and exposed style parameters may be essential while repository tool schemas are irrelevant. If the question is why a coding agent chose a particular file edit, tool definitions and workspace state may matter more than social prompt ancestry. The empirical task is to determine which fields materially improve reconstruction for which classes of work.

This proposal sharpens an important distinction between *reproducibility* and *fidelity*. A researcher can preserve every accessible receipt field and still fail to reproduce an output because sampling is stochastic, hidden infrastructure changed, the model was silently updated, or inaccessible state mattered. The receipt does not erase those limits. It makes them legible. “Unknown” becomes part of the record rather than being silently collapsed into the prompt string.

The receipt also creates a better place for provenance. Mahdavi Goloujeh and colleagues show that tracking and credit can alter willingness to share or reuse prompts [1]. Hill and colleagues’ earlier Edit Wear and Read Wear systems demonstrate a different possibility: histories of interaction can be made perceptible on the artifact itself, so that accumulated use supports sense-making [5]. SCGAI-008-A extends this idea into “prompt wear”: copied or frequently preserved phrases might become visibly polished; repeatedly mutated regions might appear abraded; abandoned branches could leave scars. Whether that visual metaphor is useful remains an empirical question. Its conceptual contribution is clearer: provenance need not be a ledger detached from the object. It can be part of the object’s actionable state.

3 A lineage edge can be more informative than either endpoint

Version-control metaphors are attractive because contemporary prompt practice increasingly resembles software maintenance. OpenAI’s current prompting guidance explicitly recommends treating production prompts as application-code-like artifacts that can be versioned, tested, reviewed, and deployed with controls [9]. Yet ordinary version control records a textual difference more reliably than it records why the difference occurred.

This matters because prompt development is often epistemic. Kirsh and Maglio distinguish pragmatic actions that advance an external task from epistemic actions that reveal information or simplify cognition [3]. A prompt execution can plausibly have either function. “Produce the final illustration” is mostly pragmatic. “Remove this modifier and see whether the lighting changes” is partly epistemic: the generated image is evidence about a hidden prompt-model relation. A generation that looks bad can therefore be a successful experiment if it falsifies a causal guess and makes the next intervention more informed. SCGAI-004-A captures this by separating artifact utility from information gain.

If prompt development contains epistemic actions, a lineage edge can preserve an experimental observation. Consider two archive records:

Ordinary diff: version 18 added the phrase “hard side light.”

Epistemic edge: version 18 added “hard side light” because version 17 failed the shadow-direction test under model M; output Y17 is the evidence; version 18 passed two of three comparison cases; confidence remains provisional.

The second record is not simply more verbose. It preserves a relation among hypothesis, intervention, evidence, and result. It gives a later user a reason to maintain, challenge, or remove the phrase. It can also reveal when the observation was version-bound. This is the distinction proposed in FIELD-20260817-04 between CREDIT PROVENANCE and EPISTEMIC PROVENANCE.

A provisional epistemic edge is:

EDGE = (from, to, actor, reason, evidence, diff, result, test, environment, confidence).

Not every prompt mutation deserves this level of capture. The practical question is which changes are consequential enough that losing their rationale creates future debugging or transfer costs. The paper therefore proposes a direct experiment: give users the same final prompt with (A) attribution-only history, (B) raw diffs, or (C) diffs enriched with reason, evidence, and result. Test whether the enriched lineage improves prediction, debugging, and transfer to a related task. If it does not, the additional archival burden is hard to justify. If it does, prompt lineage should be studied as an epistemic apparatus rather than only an authorship record.

This reframing also clarifies the community experiments described by Mahdavi Goloujeh and colleagues. One participant reports a group collectively pushing image prompts because defining literature was absent [1]. The interesting output of such activity is not merely a list of prompt tips. It is a provisional experimental tradition: hypotheses, interventions, model-specific observations, local replication, disputes, and folk causal models. DeVito’s adaptive folk theorization makes the temporal hazard explicit: such models require revision as opaque systems change [4]. A lineage that preserves only successful wording throws away the evidence needed to tell durable knowledge from fossilized ritual.

4 The archive and the active prompt should not be the same thing

Once prompt work is treated as an accumulating lineage, a new problem appears immediately: the archive grows faster than the active context can remain discriminating. A naive response is to append every successful correction to a master instruction file. The result eventually becomes its own failure mode.

OpenAI’s discussion of harness engineering describes this problem in repository agents: very large instruction manuals consume context, become stale, and make guidance harder to verify, motivating a shift toward a short map into maintained sources of truth [10]. Anthropic’s guidance for work spanning context windows similarly emphasizes external artifacts such as progress files, tests, scripts, and git history so that fresh contexts can reconstruct work state [11]. OpenAI’s prompt-caching documentation adds an infrastructural pressure: repeated exact prefixes can be reused, rewarding stable material earlier in a prompt and variable material later [15]. These are implementation-specific recommendations, but together they expose a general architecture problem.

A research archive wants to preserve surplus. It should keep failed branches, source provenance, old rationales, counterevidence, and unresolved questions because their future value is not known at capture time. An active execution context wants nearly the opposite: a small, relevant set of instructions and evidence with priorities clear enough to guide the current action. Treating one object as both archive and execution context creates what WWP-20260817-03 calls *prompt sedimentation*: historically reasonable instructions accumulate until their coexistence reduces present intelligibility.

The field therefore proposes a two-layer memory architecture:

A_t = preserved archive

C_t = RETRIEVE(A_t, goal_t, risk_t, budget_t), where C_t is a subset of A_t.

The archive and active prompt deliberately diverge. The design goal is not “make the prompt remember everything.” It is “make the work rediscoverable, and make retrieval accountable.”

WWP-20260817-04 names the resulting environmental persistence *abiding state*: continuity can survive in files, tests, logs, and addresses even when a conversational context disappears.

This architecture introduces a new failure that preservation alone cannot solve. An archive can contain exactly the correction that would prevent a mistake while the running context fails to retrieve it. FIELD-20260817-03 calls this *context recall debt*: archived evidence omitted from an active context that would have materially changed the execution. This term is again field synthesis, not source terminology.

Context recall debt is testable. Seed an archive with known critical constraints and earlier failures. Run the same tasks under different retrieval policies. When a failure occurs, perform a counterfactual replay with the omitted archived evidence included. If the failure reliably disappears, the system accumulated recall debt. This moves discussion of “memory” away from vague claims that an agent remembers or forgets and toward a measurable relation among preserved state, retrieval, and consequence.

The archive/active distinction also suggests a clean response to prompt-length maximalism. Every correction need not become permanent active prose. It can instead be promoted into one of several forms: core invariant, conditional module, executable test, retrievable source, environment state, or historical note. The important thing is not that every lesson remain in the prompt. It is that the lesson remain addressable, testable, and recoverable when relevant.

5 Words are becoming only one control surface among several

The field’s argument would be too narrow if it merely replaced a prompt string with a larger textual bundle. Current systems already make some forms of control less linguistic.

Midjourney’s Style Creator, for example, allows users to refine a reusable style reference by selecting among generated examples; its documentation distinguishes the preview text from the selection process that actually shapes the style representation [12]. WWP-20260817-05 treats this as an instance of exemplar specification: a person can recognize desired instances without possessing a complete linguistic rule for what makes them desirable. The operative control state is learned through repeated differential judgment.

At another layer, OpenAI documents prompt-generation tooling in which higher-level descriptions can be transformed into prompts or schemas using meta-prompts and related machinery [13]. Here the final wording becomes generated material. Human work can migrate upward into the task description, examples, constraints, and evaluation criteria. WWP-20260817-06 calls this *prompt compilation*: the prompt writer writes the conditions from which prompts are produced.

Model Context Protocol’s 2026 specification work provides another form: explicit handles can make application state addressable so that a model carries a reference between tool interactions rather than serializing the entire state into prose [14]. WWP-20260817-07 extracts a useful language-theoretic consequence. Natural language gains a practical companion operation: not only description, but reference. A small identifier can point to a persistent object that contains more state than the active prompt could reasonably repeat.

These examples complicate the phrase “natural-language programming.” The durable specification may live partly in words, partly in examples, partly in tests, partly in state handles, partly in repository structures, and partly in model-visible tools. The final prompt is one compiled surface among several.

This is why FIELD-20260817-02 proposes that prompt expertise is migrating from phrasing toward environment and evaluation design. The claim is not that wording stops mattering. Word order, structure, and examples plainly can matter to model behavior, and current infrastructure can make order economically consequential through caching as well [15]. The stronger claim is that excellent wording cannot compensate for an ill-specified success criterion, missing tool, stale environment, unretrieved constraint, or unrecorded model transition. As prompt text becomes easier to generate, the scarce skill may shift toward recognizing which state, tests, and evidence make the generated instruction accountable.

6 Social circulation is part of execution history

A prompt receipt would be incomplete if it recorded only machine-facing conditions. Mahdavi Goloujeh and colleagues’ most important contribution to this field is precisely the insistence that prompting has social modes of circulation [1]. A phrase may be effective because someone else discovered it. A user may refuse a reusable form because it feels appropriative. Another may strategically publish a prompt to influence downstream practice. A third may feel an obligation to “feed back” into a community that supplied useful techniques. These differences affect what becomes available for future prompting.

Two recursive source crossings make this social layer stranger.

First, Heylighen’s account of stigmergy formalizes coordination through traces left in a medium [6]. SCGAI-005-A asks whether public prompt feeds can function in this way. A user posts a prompt-output pair; a stranger sees it and reuses one fragment; a third person encounters only the descendant; coordinated conventions can accumulate without direct conversation or known ancestry. If this mechanism is real, the feed architecture is not merely a place where a community appears. It participates in producing the distribution of future prompt forms.

Second, von Hippel and von Krogh’s private-collective innovation model shows that private benefit and collective revelation need not be opposing explanations [7]. SCGAI-006-A uses this to reopen the prompt commons problem. A prompt creator may privately benefit from solving a problem, learning a model, gaining reputation, or receiving a useful output, and still reveal the technique so that a shared repertoire expands. But generative-AI ecosystems add a third beneficiary often absent from simple commons stories: the platform/model provider can also capture value from user experimentation and community knowledge production.

A receipt cannot solve the normative questions of ownership, credit, or platform appropriation. It can, however, prevent the archival system from pretending those relations do not exist. Lineage references can record known source prompts, public examples, collaborators, or communities. When origin is unknown, the receipt can say unknown. When a relationship is disputed, the archive can preserve the dispute rather than assigning a clean lineage edge by guess.

This matters for both scholarship and design. If prompting is treated as an isolated input-output event, community knowledge becomes background noise. If prompt receipts preserve social lineage where warranted, researchers can ask much sharper causal questions: Which prompt forms spread by direct collaboration? Which spread stigmergically through feeds? Which are withheld when attribution is weak? Which return improvements to their originators? Which become platform-wide conventions after apparently local experiments?

7 What a prompt receipt cannot promise

The receipt proposal has four important boundaries.

First, a receipt is not total capture. Proprietary systems can contain inaccessible instructions, routing, model weights, moderation logic, personalization, hidden sampling state, and infrastructure changes. A record that omits these cannot guarantee faithful replay. Its virtue is that it distinguishes preserved conditions from unknown ones.

Second, richer provenance creates privacy and surveillance risk. A receipt that stores conversation history, files, user identity, source prompts, or tool state can reveal material that should not travel with a research artifact. Hill et al.’s work already recognized that making interaction history visible changes the privacy properties of digital artifacts [5]. Prompt receipts therefore require disclosure boundaries, redaction rules, and perhaps hash/address references in place of source bodies.

Third, the social evidence base here is bounded. The Mahdavi Goloujeh et al. study is a qualitative study of nineteen U.S.-based Midjourney community members using English-language prompts [1]. It is valuable precisely because it opens a social unit of analysis, not because it establishes universal prompt culture.

Fourth, the infrastructure sources are particular systems. Documentation from OpenAI, Anthropic, Midjourney, and MCP demonstrates that contemporary prompt practice can involve versioning, routing, external state, generated prompts, selection-based control, explicit handles, and cached prefixes. It does not establish that all systems share one architecture. The paper’s cross-source abstractions—prompt receipt, prompt sedimentation, context recall debt, epistemic provenance—are working concepts to test, not facts inherited from those vendors.

These limits are not defects in the proposal. They specify what the next forage must discriminate.

8 A research program for reconstructable prompt practice

The field generates five immediate studies.

1. Receipt sufficiency ablation

Give independent agents or practitioners progressively richer records of the same execution: string only; string plus model/version; plus exposed parameters; plus history; plus tools/environment; plus lineage; plus tests and evaluation. Ask them to reproduce behavior and explain why later revisions occurred. The minimum bundle that materially improves reconstruction becomes task-specific evidence for receipt design.

2. Epistemic lineage transfer

Hold the final prompt constant while varying the history given to a new user: no history, raw version diffs, or reason-evidence-result enriched transitions. Test causal prediction, debugging, adaptation, and transfer to a related task. This discriminates a mere authorship ledger from an epistemic apparatus.

3. Prompt fossil archaeology

Collect falsifiable prompt rules documented for earlier named model versions. Re-run them across available versions. Classify effects as persistent, version-bound, reversed, absent, or no longer testable. This would turn the metaphor of prompt fossils into a measurable history of operational drift.

4. Context recall debt

Seed a research archive with corrections known to matter for later tasks. Compare retrieval policies. When a repeated failure occurs, replay with omitted archival evidence included. Measure the fraction of failures attributable not to absent knowledge but to failure to retrieve preserved knowledge.

5. Social trace propagation

Track distinctive prompt fragments through a public prompt environment while distinguishing direct interpersonal ties from mere exposure. If forms propagate through users who never communicate and do not know their predecessors, the stigmergic account gains support. If propagation depends strongly on direct social relations, the community-interaction account remains stronger.

These studies share a methodological advantage. They ask what changes when a specific piece of state is added, removed, or altered. That makes the boundary of “the prompt” experimentally negotiable rather than philosophically stipulated.

9 Conclusion: preserve the event, not only the incantation

The prompt string remains useful because it is visible, portable, and editable. Those virtues made it the natural unit of early prompt culture. They also make it easy to over-credit. A copied string can look self-contained even when its behavior depends on a social genealogy, a versioned model, hidden or external context, available tools, previous outputs, tests, and a sequence of epistemic interventions.

The research field assembled here points toward a different unit. A *prompt receipt* records a reconstructable execution event without claiming total access to the system. It makes provenance and environment first-class, preserves unknowns as unknowns, and provides a place to attach evaluation and change rationale. It also makes a second architectural distinction unavoidable: archival memory should be richer than active context. The field should preserve strange branches, failed tests, counterevidence, source lineage, and version history; execution should retrieve only what the present action can use.

This shift changes the practical question from “What was the prompt?” to a sequence of more discriminating questions:

What did the user submit? What context was assembled? What model and environment interpreted it? What earlier artifact or person did it inherit from? What did it produce? What did the user learn from that result? Why did the next version change? What part of this event can another researcher actually reconstruct?

Those questions are less elegant than a prompt screenshot. They are also closer to the work.

A Compiler status

This manuscript is a current wagger assembled from the zettels and sources in the accompanying SLIPCASE. It is not itself evidence. The source map names which claims depend on which zettels and sources. The exact assembly prompt is preserved separately in `prompt-receipts__2026-08-17__ASSEMBLY_APPEND` and in `_PROMPTS/000__ANCIENT_MASTER__SLIPCASE_v15.5-AM.txt`. The checkpoint records the limitation that no independent exported chat transcript was available for byte-level comparison of the pre-existing conversational zettel captures.

References

- [1] A. Mahdavi Goloujeh, A. Sullivan, and B. Magerko. 2024. *The Social Construction of Generative AI Prompts*. CHI EA '24. DOI: 10.1145/3613905.3650947.
- [2] J. Hollan, E. Hutchins, and D. Kirsh. 2000. Distributed Cognition: Toward a New Foundation for Human-Computer Interaction Research. *ACM TOCHI* 7(2):174–196. DOI: 10.1145/353485.353487.
- [3] D. Kirsh and P. Maglio. 1994. On Distinguishing Epistemic from Pragmatic Action. *Cognitive Science* 18(4):513–549. DOI: 10.1207/s15516709cog1804_1.
- [4] M. A. DeVito. 2021. Adaptive Folk Theorization as a Path to Algorithmic Literacy on Changing Platforms. *PACM HCI* 5(CSCW2), Article 339. DOI: 10.1145/3476080.
- [5] W. C. Hill, J. D. Hollan, D. Wroblewski, and T. McCandless. 1992. Edit Wear and Read Wear. *CHI '92*, 3–9. DOI: 10.1145/142750.142751.
- [6] F. Heylighen. 2016. Stigmergy as a Universal Coordination Mechanism I: Definition and Components. *Cognitive Systems Research* 38:4–13. DOI: 10.1016/j.cogsys.2015.12.002.
- [7] E. von Hippel and G. von Krogh. 2003. Open Source Software and the Private-Collective Innovation Model. *Organization Science* 14(2):209–223. DOI: 10.1287/orsc.14.2.209.14992.
- [8] M. Bolin / OpenAI. 2026. Unrolling the Codex agent loop. <https://openai.com/index/unrolling-the-codex-agent-loop/>.
- [9] OpenAI. Prompting. <https://developers.openai.com/api/docs/guides/prompting>.
- [10] OpenAI. 2026. Harness engineering: leveraging Codex in an agent-first world. <https://openai.com/index/harness-engineering/>.
- [11] Anthropic. Prompting best practices. <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>.
- [12] Midjourney. Style Creator. <https://docs.midjourney.com/hc/en-us/articles/41308374558221-Style-Creator>.
- [13] OpenAI. Prompt generation. <https://developers.openai.com/api/docs/guides/prompt-generation>.
- [14] D. Soria Parra and D. Delimarsky. 2026. The 2026-07-28 Specification. Model Context Protocol. <https://blog.modelcontextprotocol.io/posts/2026-07-28/>.
- [15] OpenAI. Prompt caching. <https://developers.openai.com/api/docs/guides/prompt-caching>.

- [16] Midjourney. Version. <https://docs.midjourney.com/hc/en-us/articles/32199405667853-Version>.